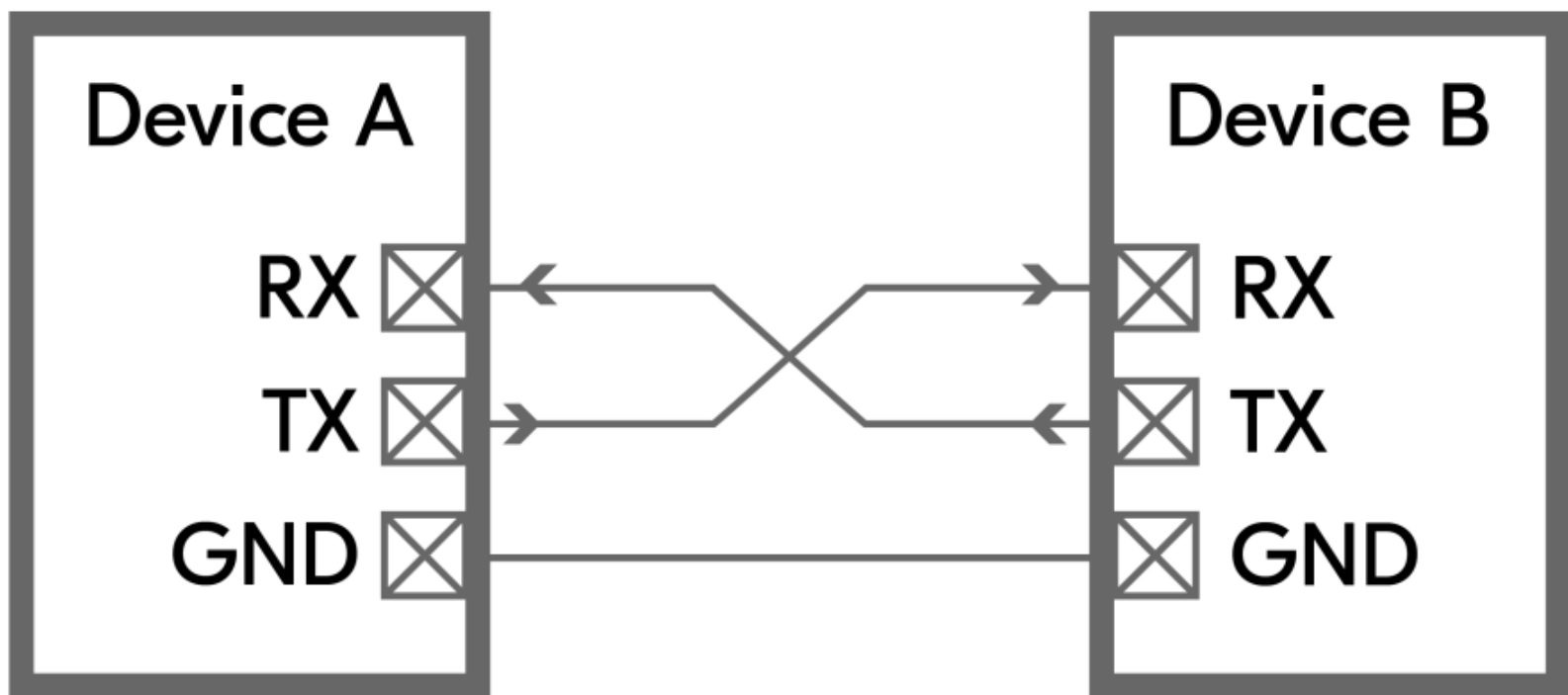


“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 12 - USART Advanced Usage



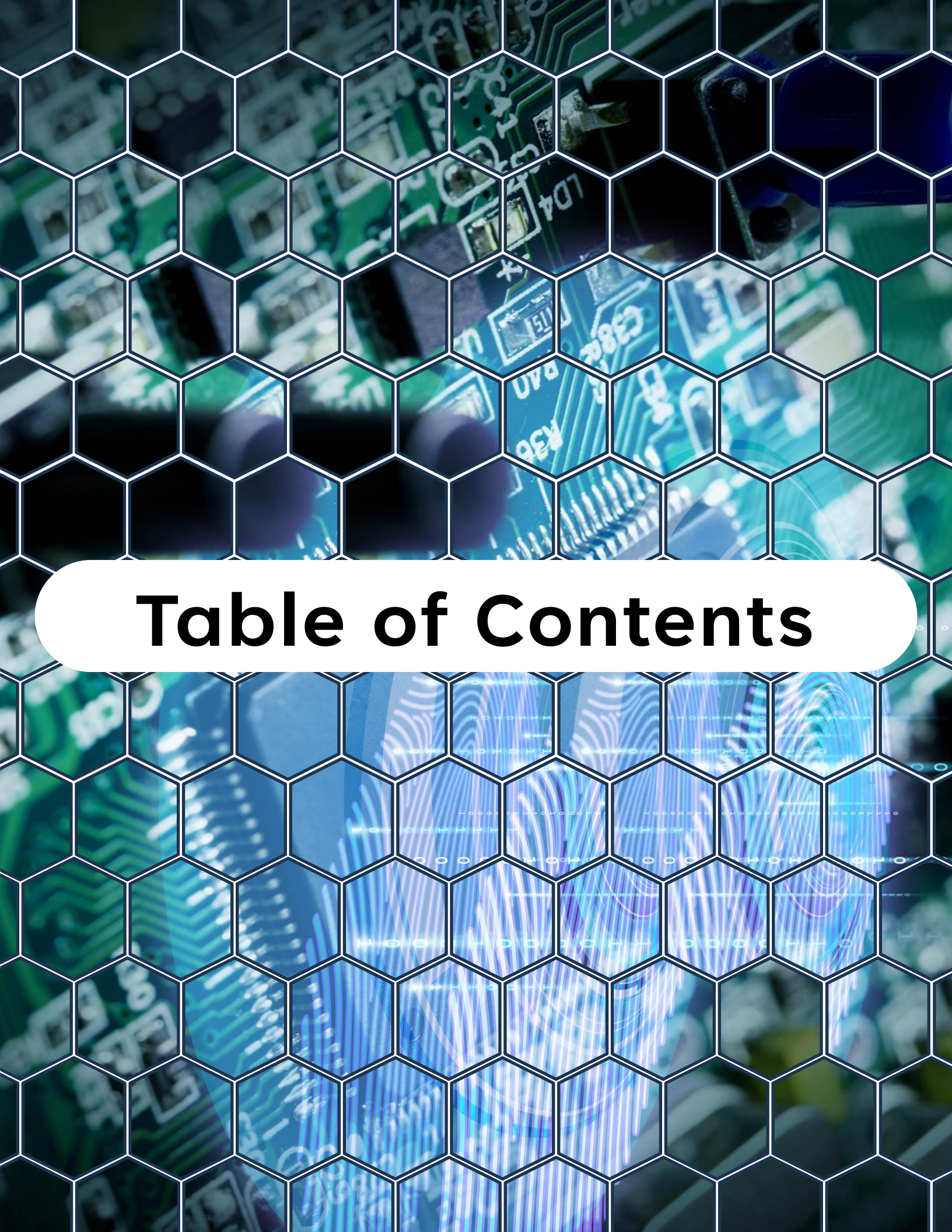
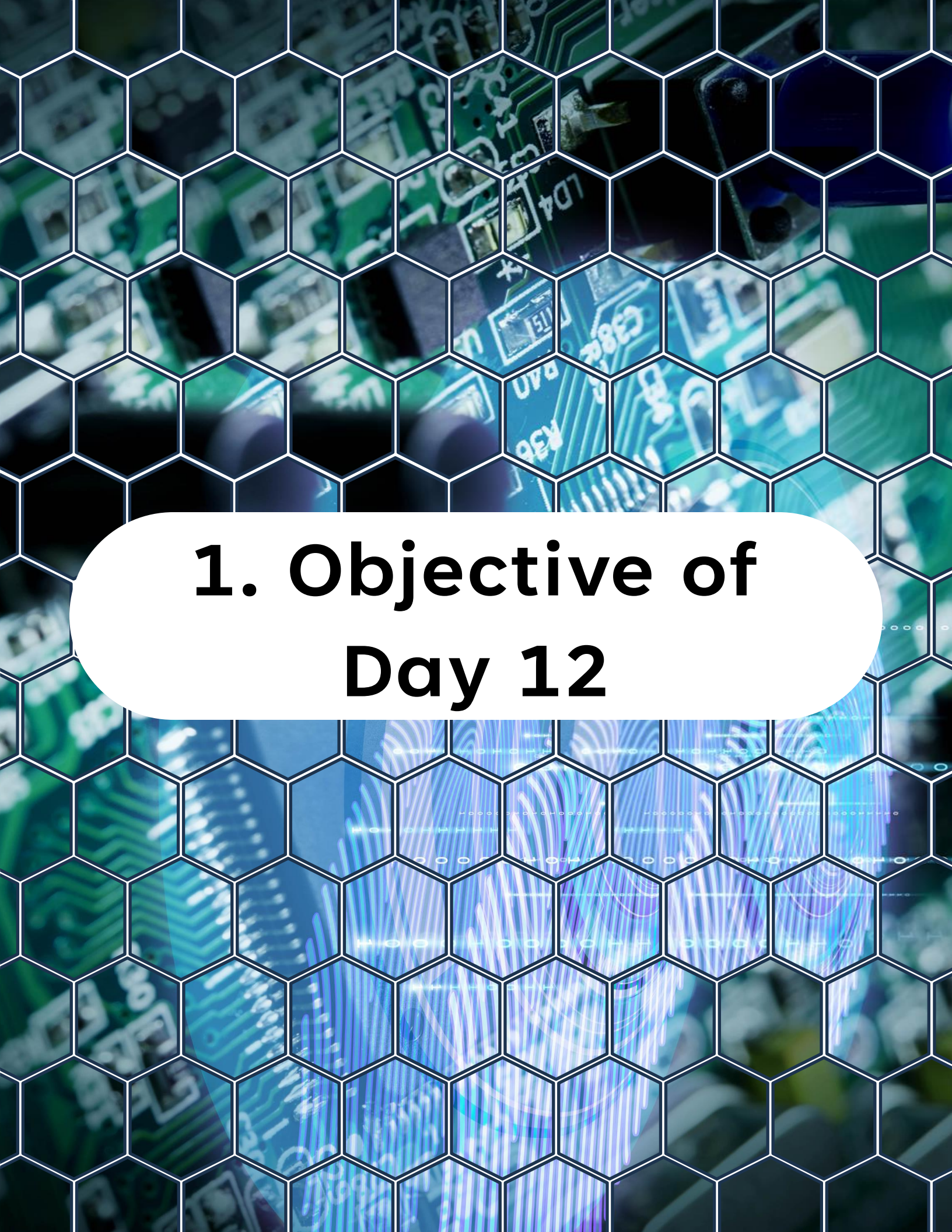


Table of Contents

Table of Contents

1. Objective of Day 12
2. Why Polling Is Not Enough
3. Interrupt-Based USART Concept
4. Ring Buffer Fundamentals
5. Buffer Definitions
6. USART Initialization with Interrupts
7. RX Interrupt Handler
8. Reading from RX Buffer (Non-Blocking)
9. TX Buffer Logic
10. TX ISR (Data Register Empty)
11. Sending a String (Non-Blocking)
12. Minimal main() Example
13. Common Design Improvements
14. Debugging Interrupt USART
15. What You Learned Today
16. What Comes Next



1. Objective of Day 12

1. Objective of Day 12

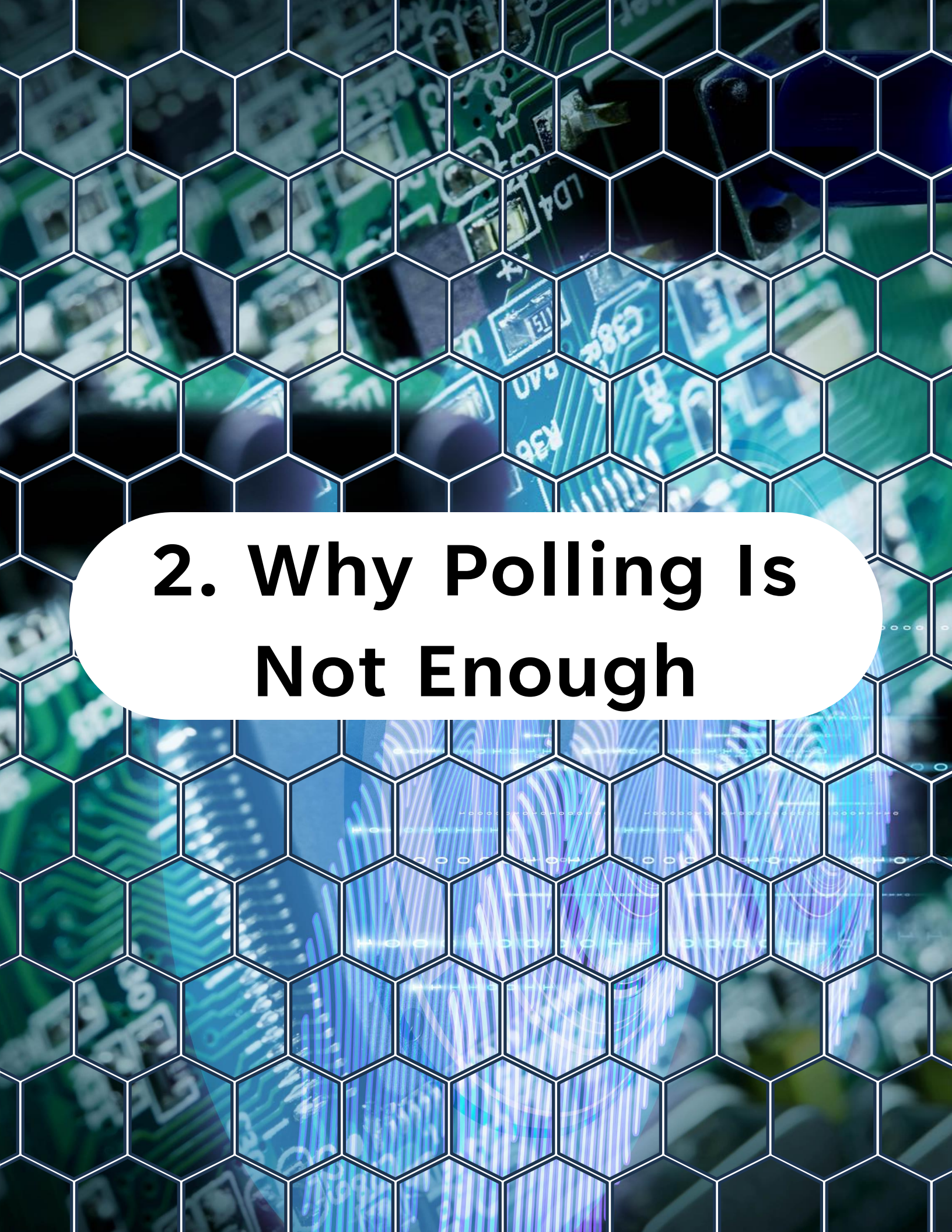
Yesterday you built a working **USART using polling**. It works, but it blocks the CPU.

Today you move to **interrupt-driven USART** with ring buffers.

By the end of this day, you will:

- Configure RX and TX interrupts
- Build a non-blocking transmit path
- Implement a ring buffer for RX
- Avoid busy-wait loops
- Build a simple command interface
- Understand how production UART drivers are structured

This is where your firmware starts to look professional.



2. Why Polling Is Not Enough

2. Why Polling Is Not Enough

Polling example from **Day 11**:



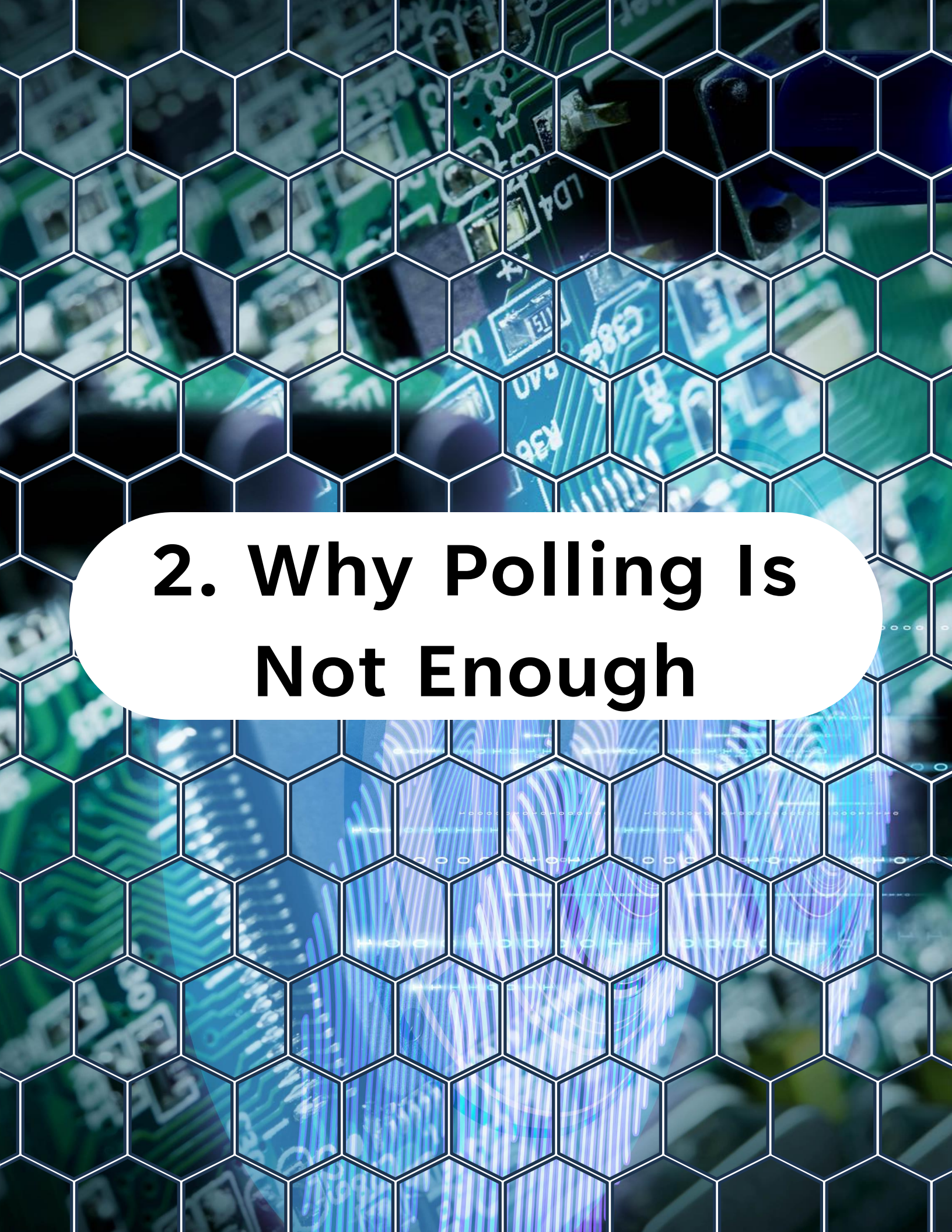
```
1 while (!(USART0.STATUS & USART_DREIF_bm));
```

This blocks the CPU. Problems with polling:

- Wastes CPU cycles
- Prevents multitasking
- Breaks timing-sensitive systems
- Does not scale

Interrupt-driven USART allows:

- CPU to do other work
- RX to capture data immediately
- TX to run in background
- Clean architecture separation



2. Why Polling Is Not Enough

2. Why Polling Is Not Enough

Polling example from **Day 11**:



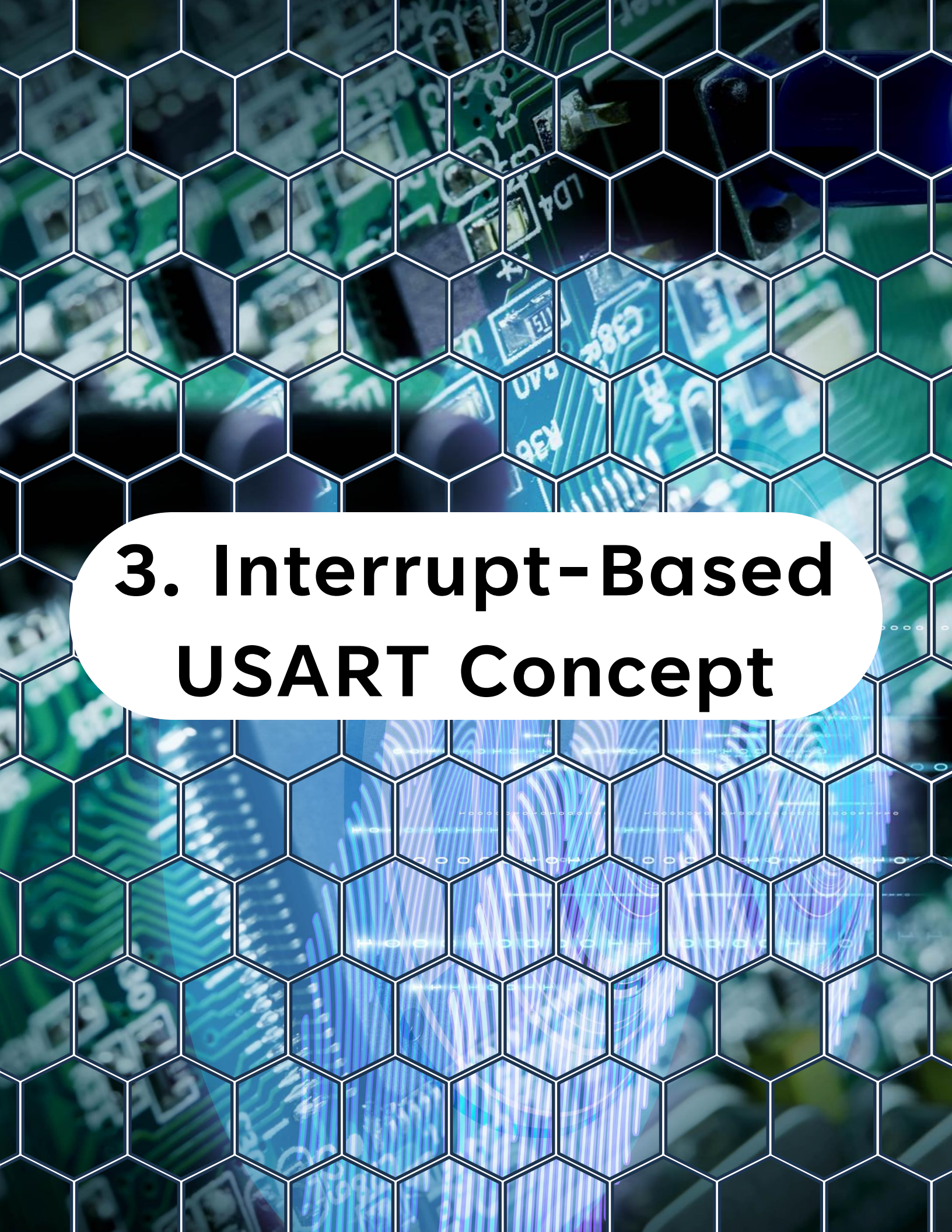
```
1 while (!(USART0.STATUS & USART_DREIF_bm));
```

This blocks the CPU. Problems with polling:

- Wastes CPU cycles
- Prevents multitasking
- Breaks timing-sensitive systems
- Does not scale

Interrupt-driven USART allows:

- CPU to do other work
- RX to capture data immediately
- TX to run in background
- Clean architecture separation



3. Interrupt-Based USART Concept

3. Interrupt-Based USART Concept

Two main interrupt types:

- RX Complete interrupt (new byte received)
- Data Register Empty interrupt (ready to send next byte)

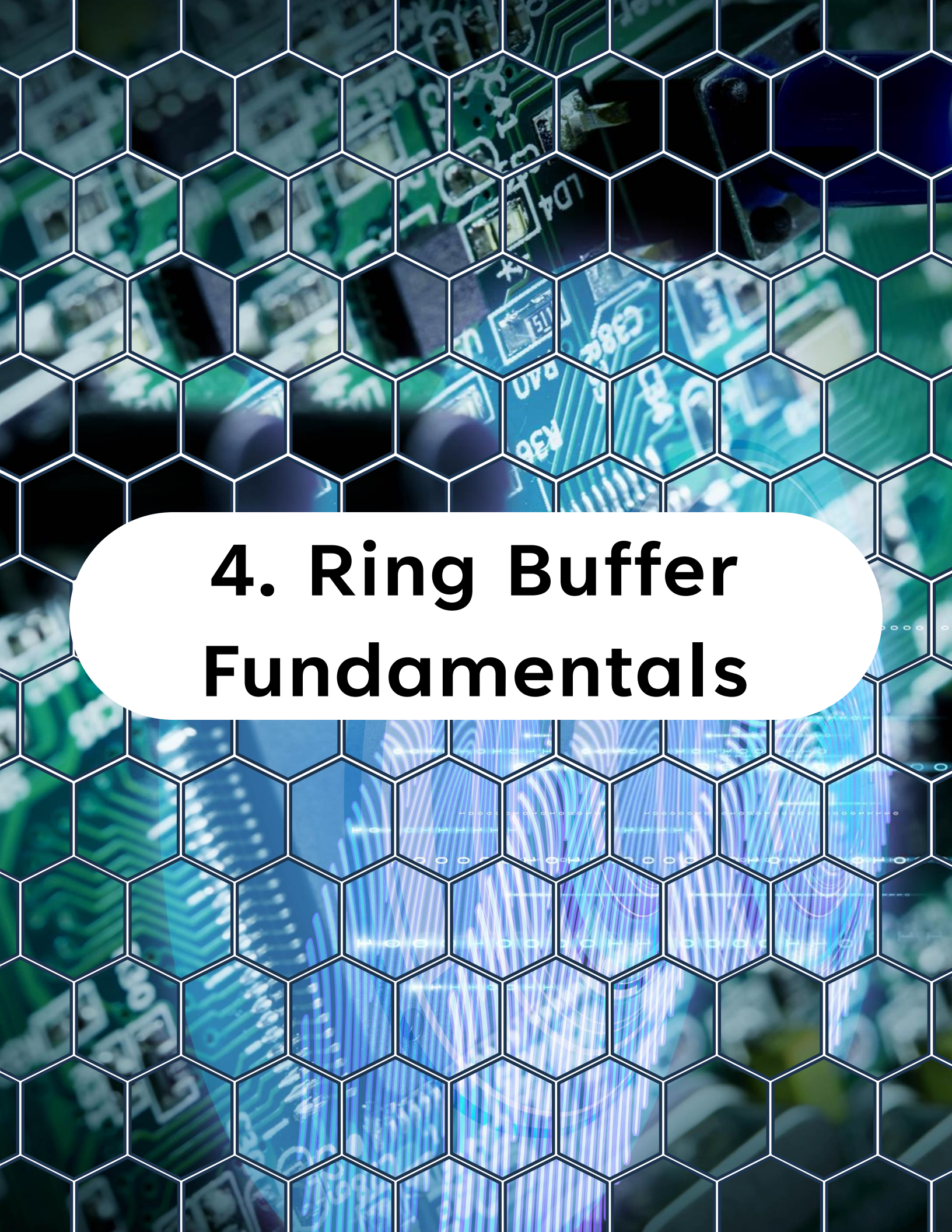
RX Flow:

- Byte arrives
- Hardware stores in **RXDATAL**
- RXC interrupt fires
- ISR pushes byte into software buffer

TX Flow:

- Software pushes data into TX buffer
- Enable DRE interrupt
- ISR sends bytes until buffer empty
- Disable DRE interrupt

This is the standard UART driver architecture.



4. Ring Buffer Fundamentals

4. Ring Buffer Fundamentals

A ring buffer (circular buffer) is:

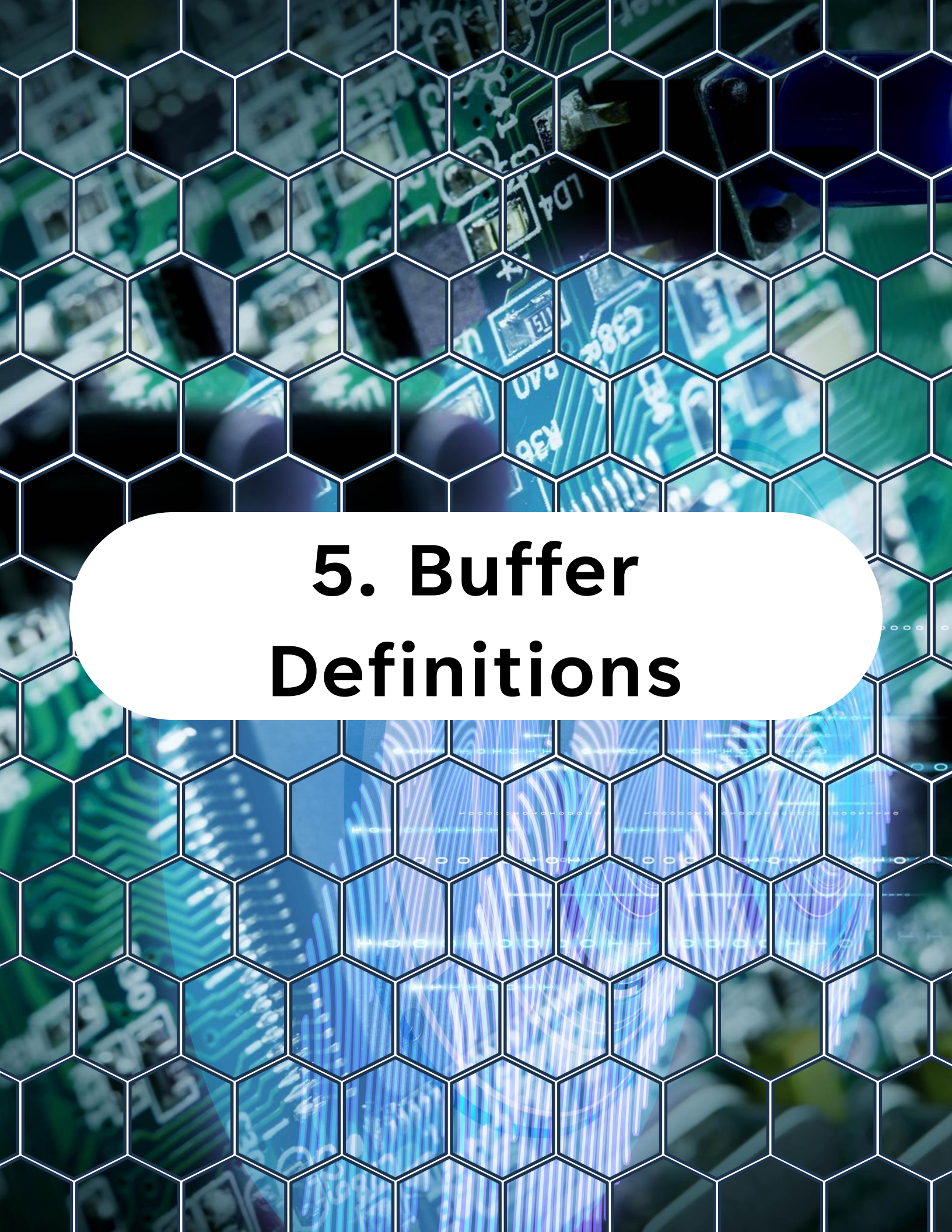
- Fixed-size array
- Head index (write position)
- Tail index (read position)
- Wraps automatically

Why ring buffers?

- Efficient
- No shifting memory
- Perfect for streaming data

We will create:

- RX buffer (for incoming data)
- TX buffer (for outgoing data)



5. Buffer Definitions

5. Buffer Definitions

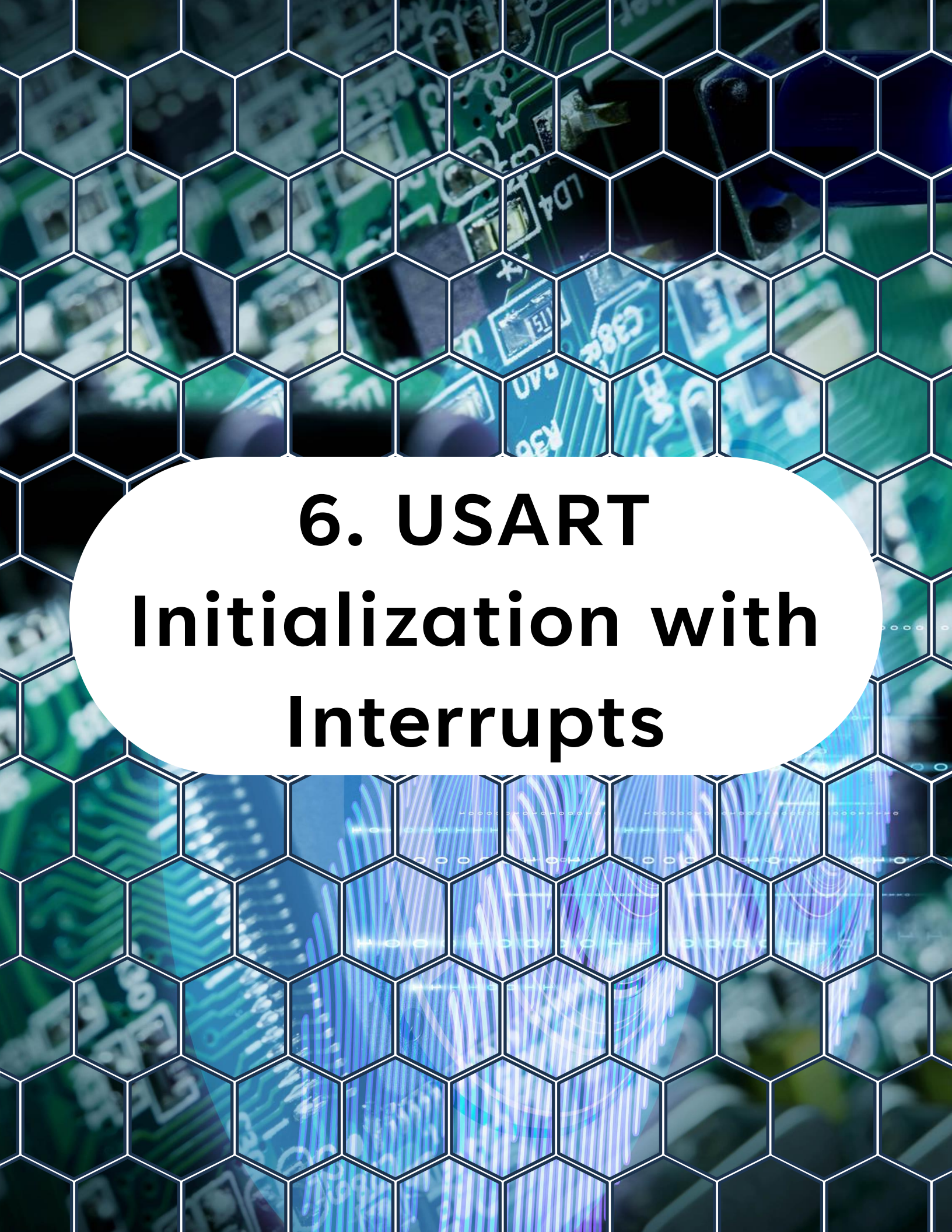
Buffers example configuration:



```
1  #define UART_RX_BUFFER_SIZE    64
2  #define UART_TX_BUFFER_SIZE    64
3
4  static volatile uint8_t rx_buffer[UART_RX_BUFFER_SIZE];
5  static volatile uint8_t tx_buffer[UART_TX_BUFFER_SIZE];
6
7  static volatile uint8_t rx_head = 0;
8  static volatile uint8_t rx_tail = 0;
9
10 static volatile uint8_t tx_head = 0;
11 static volatile uint8_t tx_tail = 0;
```

We use volatile because:

- These variables are modified inside ISRs.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white oval contains the title text in a bold, black, sans-serif font.

6. USART Initialization with Interrupts

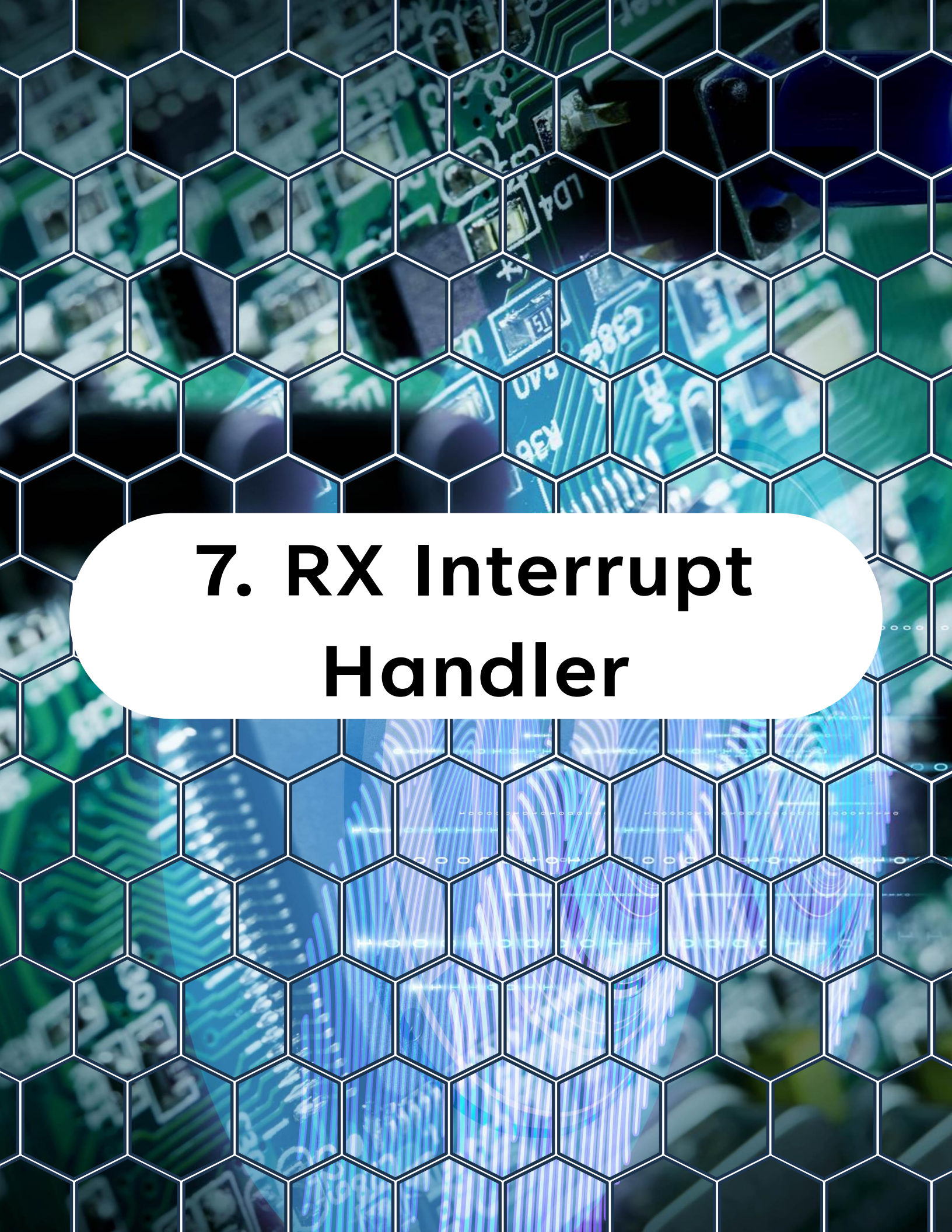
6. USART Initialization with Interrupts

We extend Day 11 configuration:

```
1 static void usart0_init(void)
2 {
3     // Configure pins
4     PORTA.DIRSET = PIN0_bm; /* TX */
5     PORTA.DIRCLR = PIN1_bm; /* RX */
6
7     // Configure baud rate
8     USART0.BAUD = USART0_BAUD_VALUE;
9
10    // Configure frame format: 8 data bits, no parity, 1 stop bit
11    USART0.CTRLA = USART_CHSIZE_8BIT_gc // 8-bit character size
12                | USART_PMODE_DISABLED_gc // No parity
13                | USART_SBMODE_1BIT_gc; // 1 stop bit
14
15    // Enable Receive Complete Interrupt
16    // This interrupt fires when a byte is received
17    USART0.CTRLA = USART_RXCIE_bm;
18
19    // Enable transmitter and receiver
20    USART0.CTRLB = USART_TXEN_bm | USART_RXEN_bm;
21 }
```

Notice:

- We enable RX Complete Interrupt (**RXCIE**)
- We do NOT enable **DRE** interrupt yet



7. RX Interrupt Handler

7. RX Interrupt Handler

When a byte arrives:

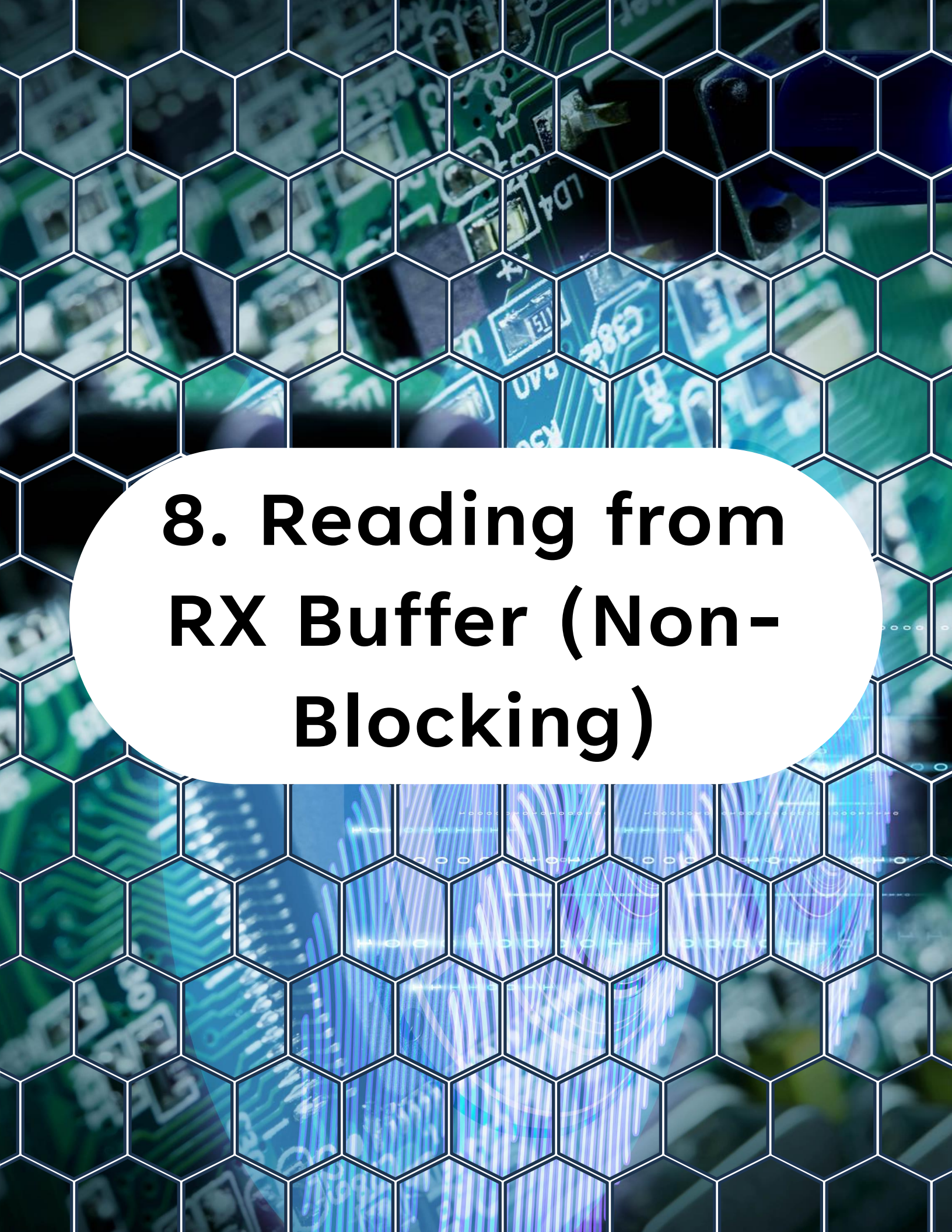
- ISR reads **RXDATAL**
- Stores it in ring buffer
- Advances head

```
1  ISR(USART0_RXC_vect)
2  {
3      // Read the received byte from the USART data register
4      uint8_t data = USART0.RXDATAL;
5
6      // Calculate the next head index in the circular buffer
7      uint8_t next_head = (rx_head + 1) % UART_RX_BUFFER_SIZE;
8
9      // Check if the buffer is not full before storing
10     // the received byte
11     if (next_head != rx_tail)
12     {
13         rx_buffer[rx_head] = data;
14         rx_head = next_head;
15     }
16     /* else: buffer overflow, byte discarded */
17 }
```

7. RX Interrupt Handler

Important:

- Always read **RXDATA1** inside ISR
- Avoid heavy processing inside ISR
- Only move data into buffer



8. Reading from RX Buffer (Non- Blocking)

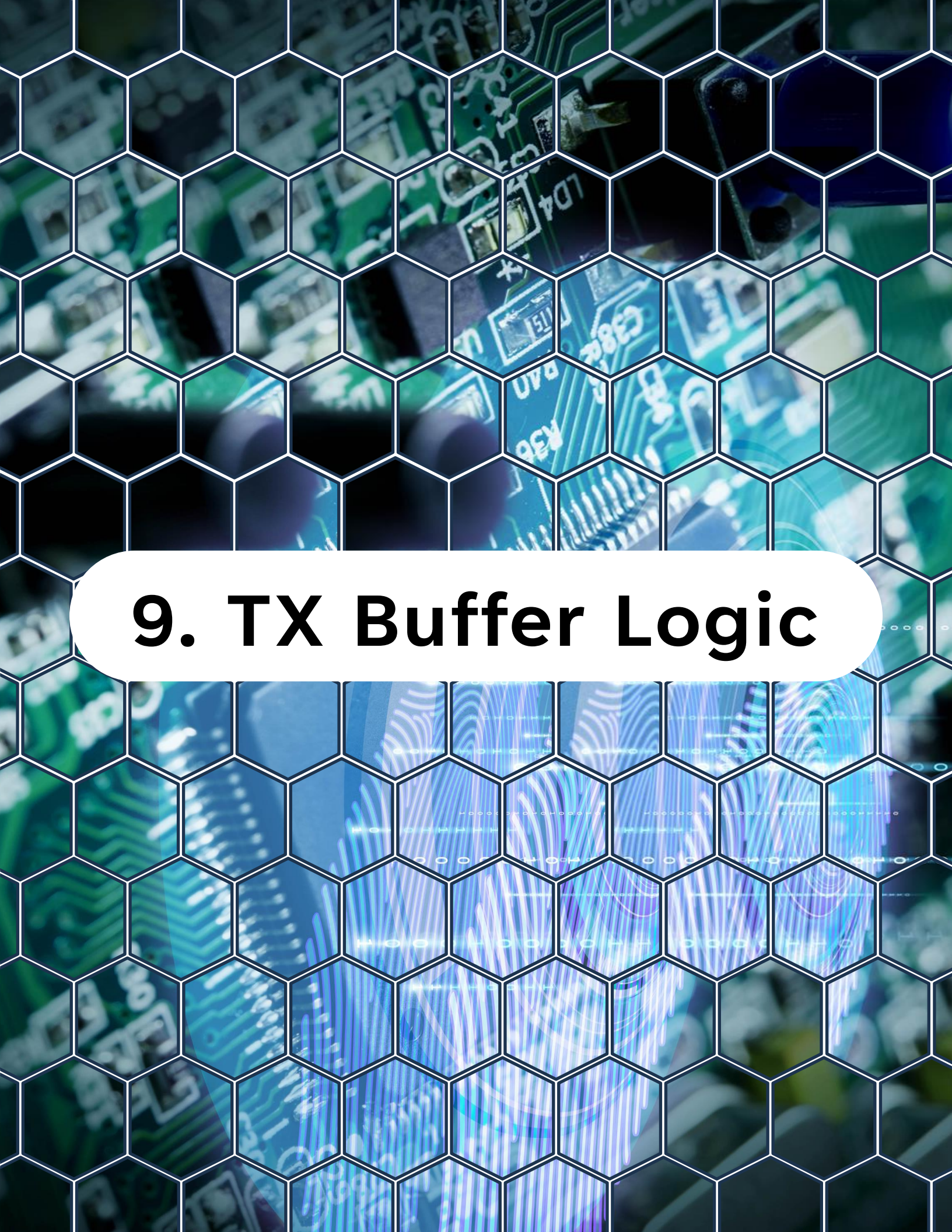
8. Reading from RX Buffer (Non-Blocking)

```
1 static uint8_t usart0_available(void)
2 {
3     return (rx_head != rx_tail);
4 }
5
6 static uint8_t usart0_read(void)
7 {
8     if (rx_head == rx_tail)
9     {
10         return 0; /* no data */
11     }
12
13     // Read a byte from the buffer
14     uint8_t data = rx_buffer[rx_tail];
15     rx_tail = (rx_tail + 1) % UART_RX_BUFFER_SIZE;
16
17     return data;
18 }
```

Now main loop can check:

```
1 if (usart0_available())
2 {
3     uint8_t c = usart0_read();
4 }
```

No blocking. Clean.



9. TX Buffer Logic

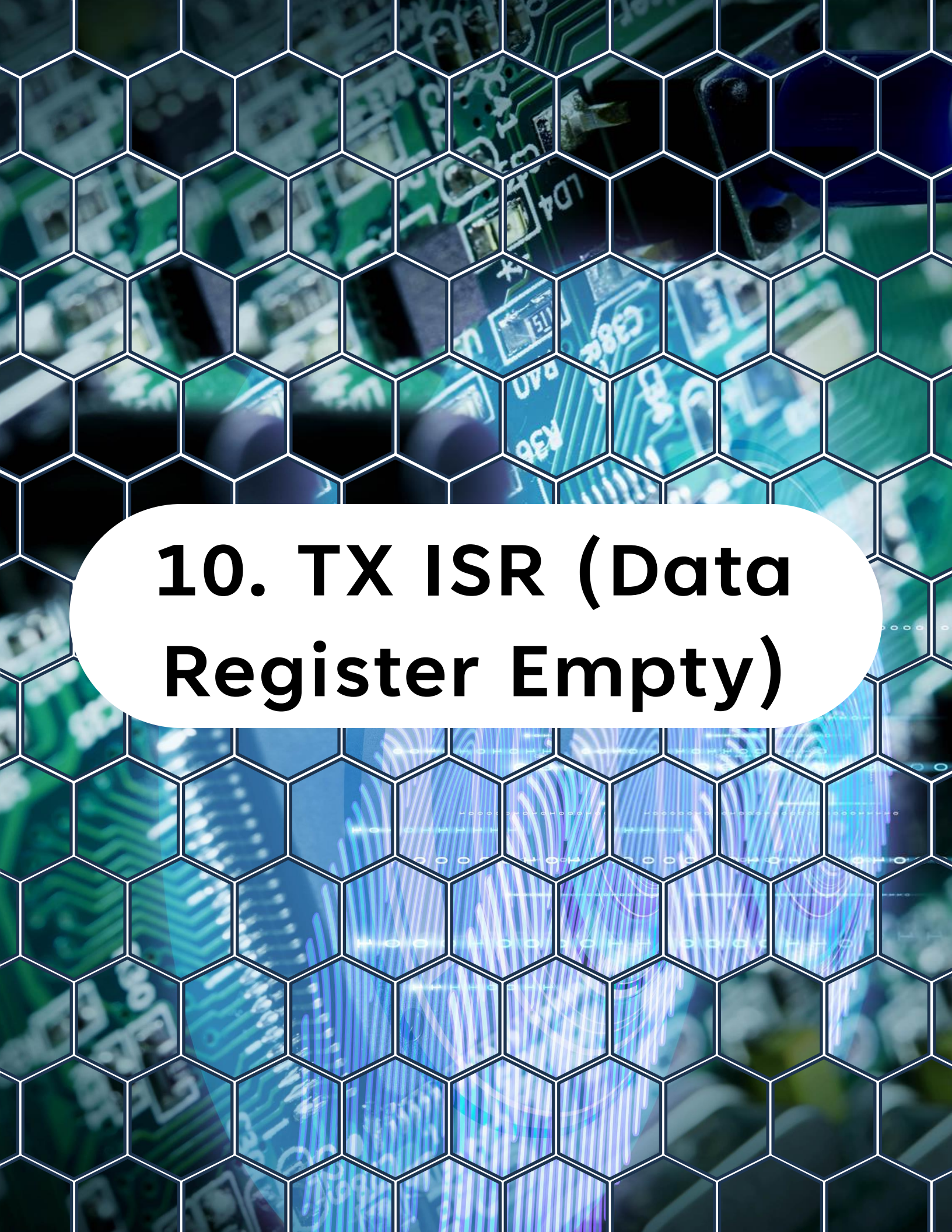
9. TX Buffer Logic

To send data:

- Push into TX buffer
- Enable DRE interrupt
- ISR handles actual sending

TX Write Function

```
1 static void usart0_write(uint8_t data)
2 {
3     uint8_t next_head = (tx_head + 1) % UART_TX_BUFFER_SIZE;
4
5     while (next_head == tx_tail)
6     {
7         /* Wait or handle overflow */
8     }
9
10    tx_buffer[tx_head] = data;
11    tx_head = next_head;
12
13    /* Enable Data Register Empty interrupt */
14    USART0.CTRLA |= USART_DREIE_bm;
15 }
```

10. TX ISR (Data Register Empty)

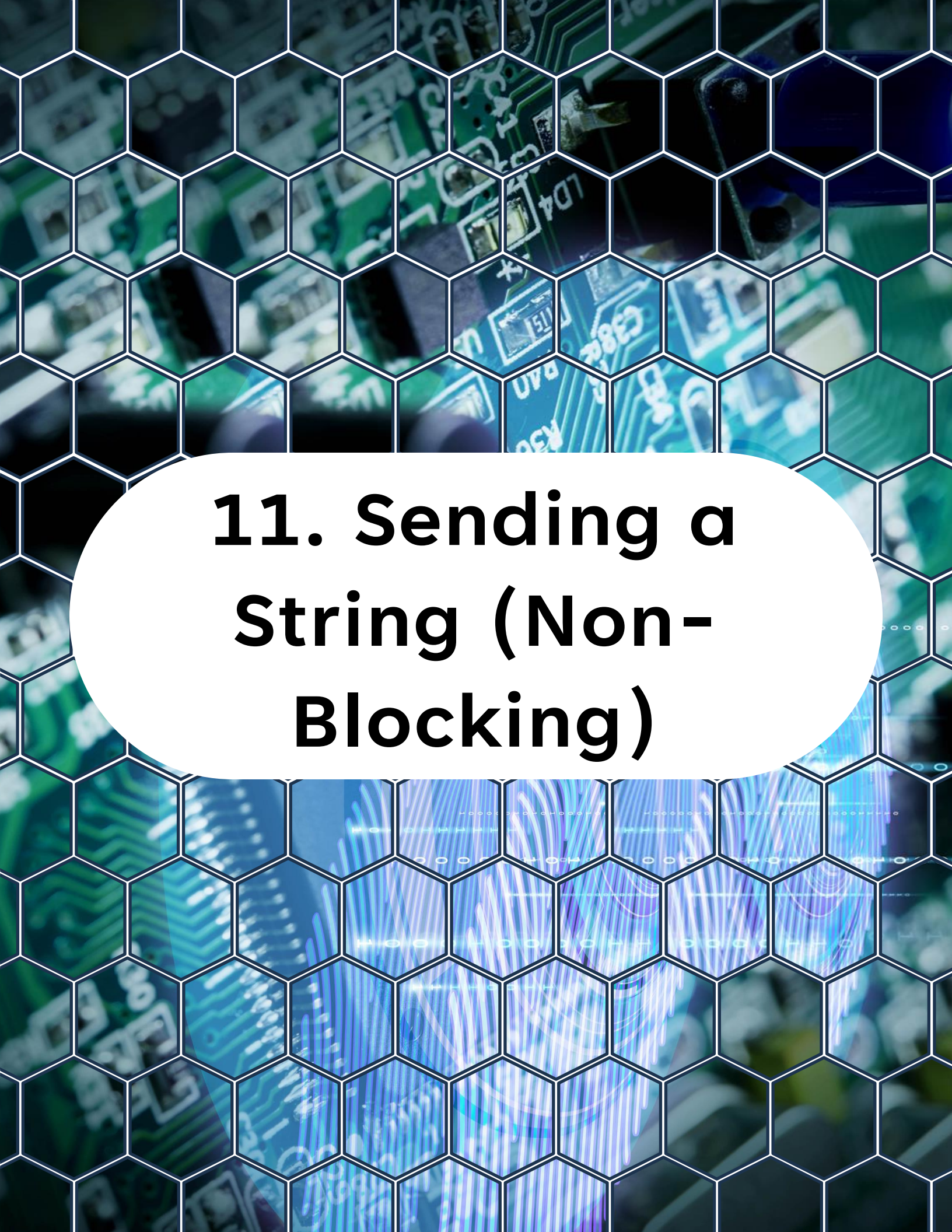
10. TX ISR (Data Register Empty)

```
1  ISR(USART0_DRE_vect)
2  {
3      if (tx_head == tx_tail)
4      {
5          /* Buffer empty, disable interrupt */
6          USART0.CTRLA &= ~USART_DREIE_bm;
7      }
8      else
9      {
10         USART0.TXDATA = tx_buffer[tx_tail];
11         tx_tail = (tx_tail + 1) % UART_TX_BUFFER_SIZE;
12     }
13 }
```

Behavior:

- When TX register empty, ISR fires
- Sends next byte
- If buffer empty, interrupt disabled

This is efficient and fully non-blocking.



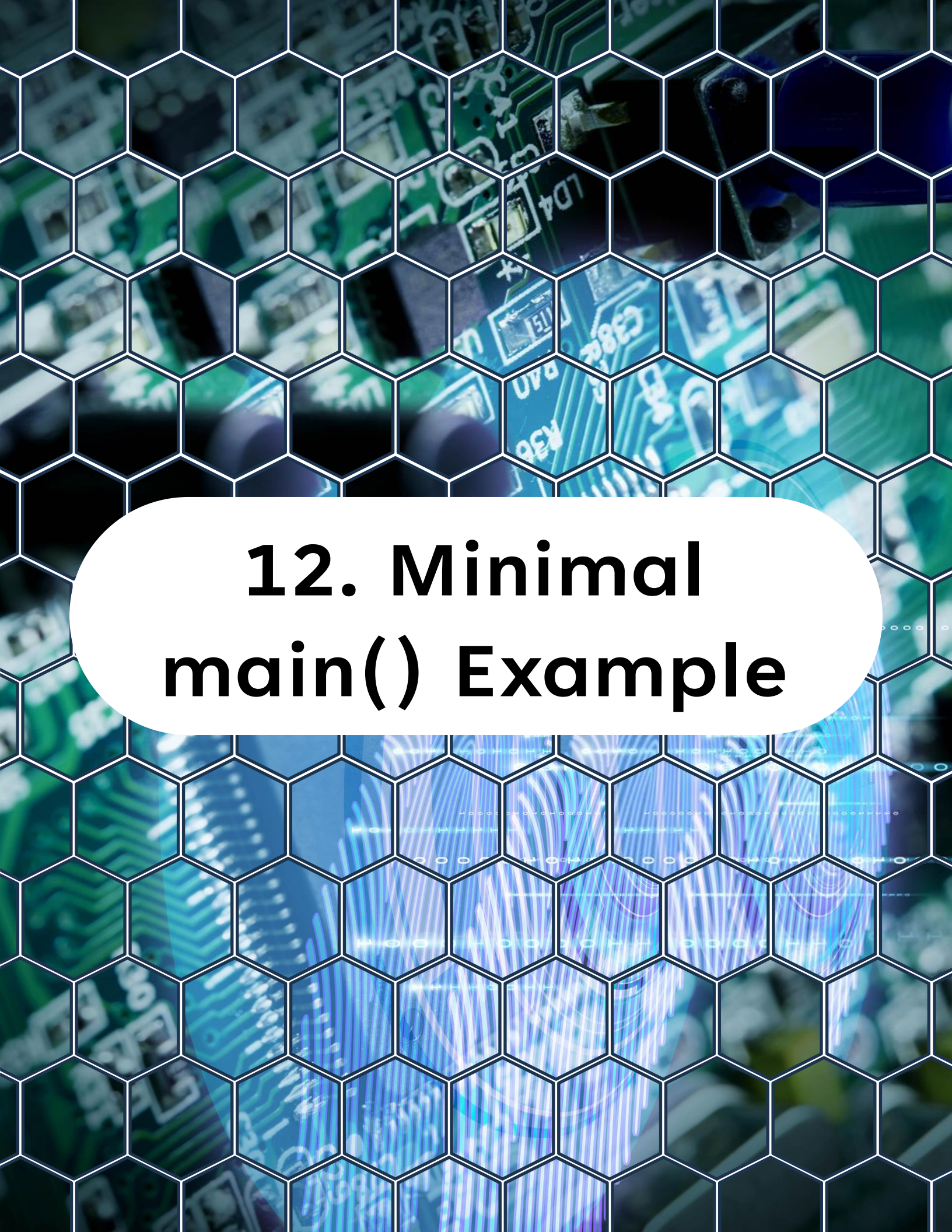
11. Sending a String (Non- Blocking)

11. Sending a String (Non-Blocking)



```
1 static void usart0_write_string(const char *str)
2 {
3     while (*str)
4     {
5         usart0_write((uint8_t)(*str));
6         str++;
7     }
8 }
```

No busy wait on hardware flags. Only buffer management.



12. Minimal main() Example

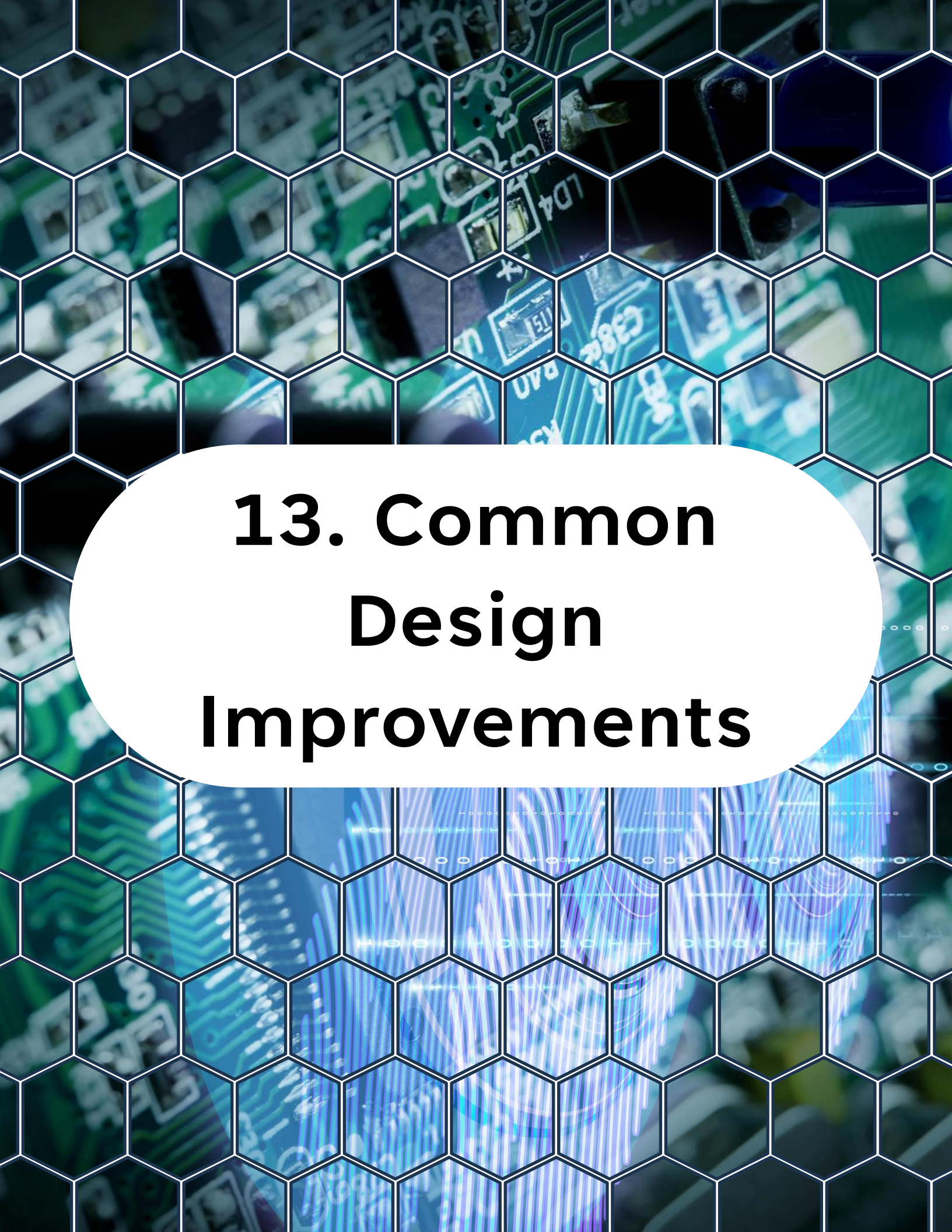
12. Minimal main() Example

```
1  int main(void) {
2
3      usart0_init();
4      sei();
5
6      usart0_write_string("USART Interrupt Mode Ready\r\n");
7
8      while (1) {
9          if (usart0_available()) {
10             uint8_t c = usart0_read();
11
12             /* Simple command: toggle LED on 't' */
13             if (c == 't') {
14                 PORTA.OUTTGL = PIN0_bm;
15                 usart0_write_string("LED toggled\r\n");
16             }
17         }
18     }
19 }
```

Now:

- RX is interrupt-driven
- TX is interrupt-driven
- Main loop stays responsive

This is real embedded design.



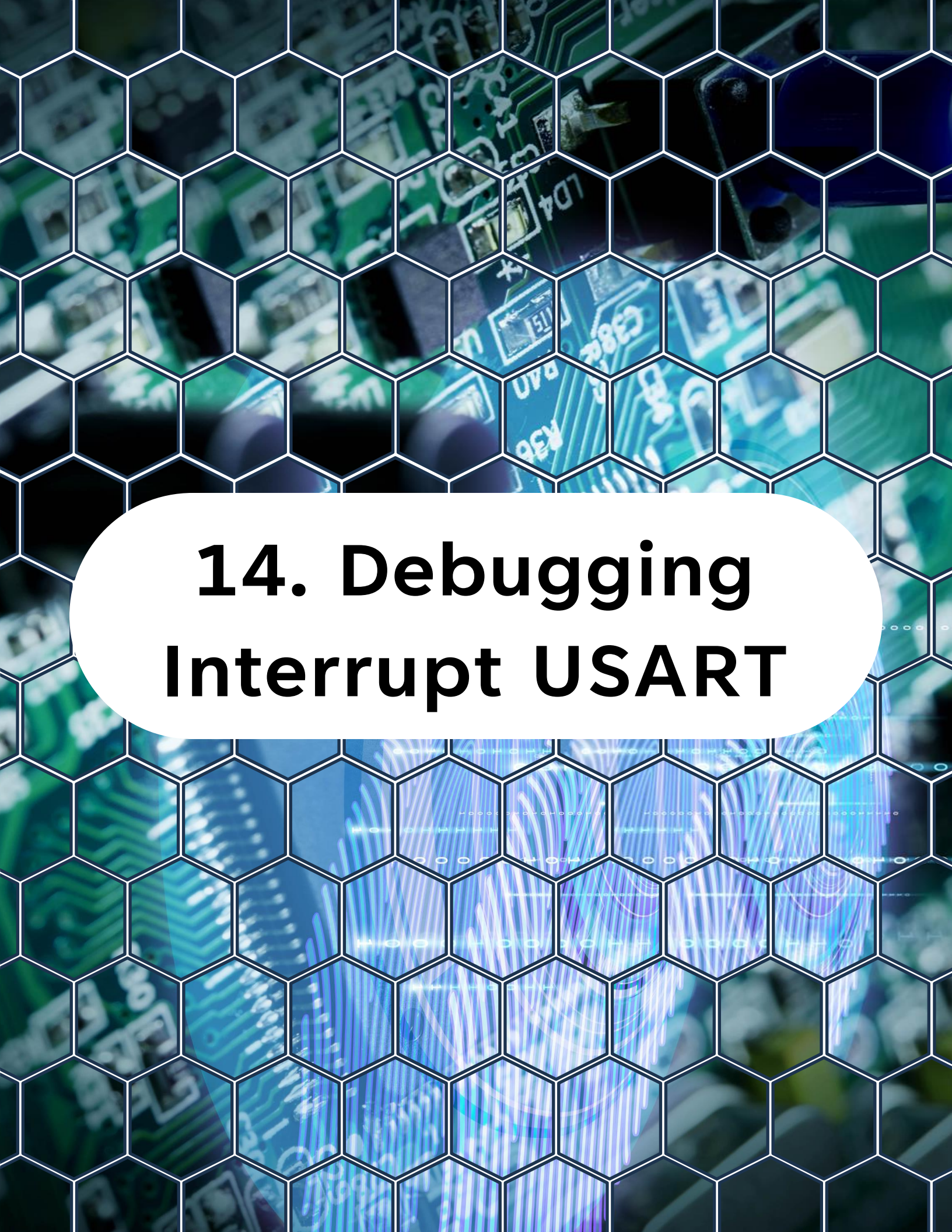
13. Common Design Improvements

13. Common Design Improvements

Professional enhancements:

- Add overflow detection flag
- Use power-of-two buffer sizes for faster wrap
- Protect shared data if using multi-byte variables
- Add line-based command parsing

But what you built today is already production-capable.



14. Debugging Interrupt USART

14. Debugging Interrupt USART

If RX ISR never triggers:

- Check USART_RXCIE_bm enabled
- Check global interrupts enabled
- Check correct vector name
- Confirm baud rate correct

If TX stops:

- Confirm DRE interrupt enabled
- Confirm ISR clears properly
- Check buffer indexes

If data corrupted:

- Check baud rate math
- Check clock configuration
- Check buffer overflow



15. What You Learned Today

15. What You Learned Today

By the end of Day 12, you understand:

- Interrupt-driven USART architecture
- Ring buffer implementation
- Non-blocking TX and RX
- Clean ISR design principles
- Production-ready serial communication patterns

This is a major upgrade over polling.



16. What Comes Next

16. What Comes Next

Day 13 - SPI Master Mode

Next, you will:

- Configure SPI in master mode
- Understand clock polarity and phase
- Transfer data to an external device
- Build a basic SPI driver

From this point on, your MCU will communicate not only with PCs, but with other hardware devices.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.

Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>